Name: Muhammad Talha Javed
Subject : Computer Architecture
Assignment : Direct-Mapped Cache with Page-Table Translation (No TLB)
Submitted to : Sir Muhammad Yousaf

Chapter 1: Introduction

1.1 Introduction

Cache memory is used to reduce the average memory access time by storing frequently accessed data closer to the processor. Before accessing the cache, the processor-generated virtual address must be translated into a physical address. In this project, the address translation is performed using a **page-table based mechanism without a Translation Lookaside Buffer (TLB)**. Therefore, every processor memory request first accesses the page-table RAM before the cache lookup begins.

This project implements a **direct-mapped cache controller** integrated with a page-table based address translation unit using Verilog HDL. The design supports virtual-to-physical address translation, cache hit and cache miss detection, page hit and page fault handling, and communication with main memory. The complete system is verified through simulation using different memory access scenarios specified in the assignment.

1.2 Objectives

The objectives of this project are:

- Design a direct-mapped cache controller.
- Implement virtual-to-physical address translation using page tables.
- Generate page hit and page fault signals.
- Perform cache read and write operations correctly.
- Interface the cache controller with main memory.
- Verify the complete system using simulation for page hits, page faults, cache hits, cache misses, and conflict misses.

1.3 Given System Specifications

The cache controller is implemented according to the specifications provided in the assignment.

Parameter	Value
Cache Organization	Direct Mapped
Number of Cache Lines	8
Number of Main Memory Lines	32
Cache Line Size	128 bits (16 Bytes)
Page Size	32 Bytes (2 Cache Lines)
Words per Page	8
CPU Data Width	32 bits
Address Bus Width	32 bits (7 effective virtual bits, 9 effective physical bits)

1.4 Report Organization

This report first presents the address translation mechanism, including virtual and physical address calculations, page-table implementation, and page fault handling. It then describes the cache controller design, cache datapath, controller operation, and simulation results. Finally, the report verifies that the implemented design satisfies the assignment requirements.

Chapter 2: Address Translation Design

2.1 Page Offset Calculation

The page size given in the assignment is **32 bytes**, and each cache line stores **16 bytes**. Therefore, one page contains exactly **two cache lines**. The page offset must identify the cache line within the page, the word within the cache line, and the byte within the selected word.

Calculation

- Line within page = **1 bit**
- Word offset = **2 bits**
- Byte offset = **2 bits**

Therefore,

Page Offset = 1 + 2 + 2 = 5 bits

Since,

$$2^5 = 32 \text{ bytes}$$

the calculated page size matches the required page size of **32 bytes**.

Table 2.1 Page Offset Calculation

Field	Bits
Line within Page	1
Word Offset	2
Byte Offset	2
Total Page Offset	**5 bits**

2.2 Virtual Address Format

The processor generates a **32-bit virtual address**, but only the lower **7 bits** are used in this project. These 7 bits are divided into a **2-bit Virtual Page Number (VPN)** and a **5-bit Page Offset**. The remaining upper bits are reserved and kept at zero.

Effective Virtual Address Calculation

Number of virtual pages per process = **4**

Therefore,

$$VPN = \log_2(4) = 2 \text{ bits}$$

Page Offset = **5 bits**

Hence,

$$\boxed{\text{Effective Virtual Address Width} = 2 + 5 = 7 \text{ bits}}$$

Figure 2.2: *Virtual Address Format (Insert virtual address diagram here.)*

2.3 Physical Address Format

Main memory consists of **32 memory lines**, each storing **16 bytes**. Since each page contains **two cache lines**, the physical memory contains:

$$\frac{32}{2} = 16 \text{ page frames}$$

Therefore,

$$PFN = \log_2(16) = 4 \text{ bits}$$

The physical address is formed by concatenating the **4-bit Physical Frame Number (PFN)** with the **5-bit Page Offset**.

$$\boxed{\text{Physical Address} = PFN \;\; || \;\; Page\ Offset}$$

Hence,

$$\boxed{\text{Effective Physical Address Width} = 4 + 5 = 9 \text{ bits}}$$

Table 2.2 Physical Address Fields

Field	Bits
Physical Frame Number (PFN)	4
Page Offset	5
Total Physical Address	**9 bits**

Figure 2.3: *Physical Address Format (Insert physical address diagram here.)*

2.4 Page Table Organization

The system implements **two page tables**, one for each processor. Each page table contains **four Page Table Entries (PTEs)**. The processor ID selects the corresponding **Page Table Base Register (PTBR)**. The selected PTBR is then added to the Virtual Page Number (VPN) to generate the address of the required Page Table Entry.
The Page Table Entry stores:
Valid bit (1 bit)

- Physical Frame Number (4 bits)

Thus, the minimum PTE width is **5 bits**.

2.5 PTBR Selection

To support multiple processes, the system implements two **Page Table Base Registers (PTBRs)**. The processor ID determines which page table is accessed during address translation. Processor 0 uses **PTBR0**, while Processor 1 uses **PTBR1**. This allows each processor to maintain its own virtual-to-physical address mapping.

Table 2.4 PTBR Selection

Processor ID	Selected PTBR	RAM Entries
0	PTBR0 = 0	0–3
1	PTBR1 = 4	4–7

RTL Snippet

```
assign selected_ptbr = (processor_id == 1'b0) ? 3'd0 : 3'd4;
```

The selected PTBR is added to the Virtual Page Number (VPN) to locate the required Page Table Entry (PTE).

2.6 Page Table RAM

The page-table RAM stores both page tables in a single memory. Each page table contains four entries, resulting in a total of eight Page Table Entries (PTEs). Every PTE contains a **Valid bit** and a **Physical Frame Number (PFN)**.

The PTE format is shown below.

Table 2.5 Page Table Entry Format

Field	Bits
Valid	1
PFN	4
Total 5 bits	

During address translation, the selected PTE is read from the RAM and checked to determine whether the requested page is valid.

2.7 PTE Address Calculation

The address of the required Page Table Entry is obtained by adding the selected PTBR to the Virtual Page Number.

$$\boxed{\text{PTE Address} = \text{Selected PTBR} + \text{VPN}}$$

Example

For **Processor 1**:

- PTBR = 4
- VPN = 2

Therefore,

$$\text{PTE Address} = 4 + 2 = 6$$

Hence, RAM location **6** is accessed to obtain the corresponding Page Table Entry.

RTL Snippet

```
assign page_table_address = selected_ptbr + vpn;
```

2.8 Page Hit and Page Fault

After the selected Page Table Entry is read from RAM, the Valid bit determines whether the translation is successful.

- If **Valid = 1**, a **Page Hit** occurs.
- If **Valid = 0**, a **Page Fault** occurs.

During a page fault, the cache controller is not allowed to access the cache because no valid physical address is available.

RTL Snippet

```
assign page_hit   = valid;
assign page_fault = ~valid;
```

2.9 Physical Address Generation

When a page hit occurs, the Physical Frame Number (PFN) obtained from the Page Table Entry is concatenated with the original Page Offset to generate the physical address.

$$\boxed{\text{Physical Address} = \text{PFN} \;||\; \text{Page Offset}}$$

Example

PFN = **0111**

Page Offset = **10100**

Therefore,

$$\text{Physical Address} = 0111 \;||\; 10100 = 011110100$$

This translated physical address is then forwarded to the cache controller for cache lookup.

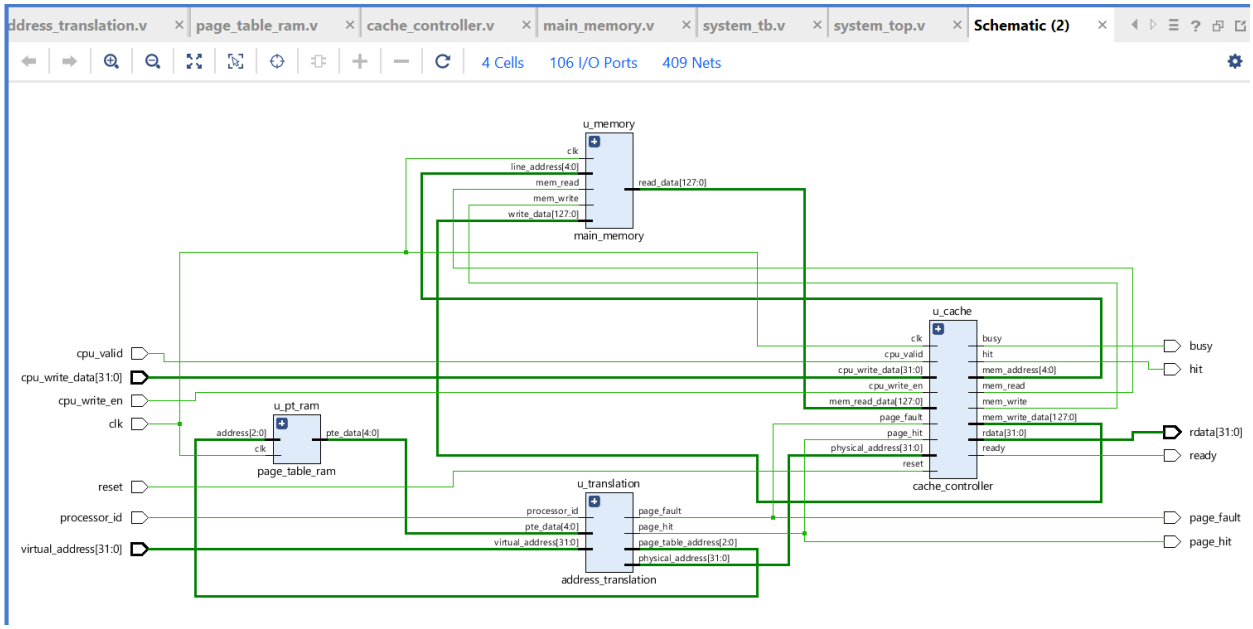
RTL Snippet

```
assign physical_address = {23'b0, pfn, page_offset};
```

2.10 Cache Blocking During Page Fault

If the selected Page Table Entry is invalid, the address translation unit asserts the **page_fault** signal. In this condition, the translated physical address is not forwarded to the cache controller, preventing any cache or main-memory access. Instead, the system reports a page fault and terminates the current memory request. This behavior ensures that memory accesses are performed only for valid virtual pages

Chapter 3: Cache Controller Design



3.1 Cache Address Fields

The cache controller receives the **translated physical address** from the Address Translation Unit instead of the original virtual address. Since the cache is direct-mapped and contains **8 cache lines**, the physical address is divided into four fields: **Tag**, **Index**, **Word Offset**, and **Byte Offset**.

Cache Address Calculation

Physical Address Width = **9 bits**

Cache Lines = **8**

$$\text{Index Bits} = \log_2(8) = 3 \text{ bits}$$

Cache Line Size = **16 Bytes**

$$\text{Block Offset} = \log_2(16) = 4 \text{ bits}$$

The block offset consists of:

- Word Offset = 2 bits

- Byte Offset = 2 bits

Therefore,

$$\text{Tag Bits} = 9 - 3 - 4 = 2 \text{ bits}$$

Table 3.1 Physical Address Fields

Address	Number of Bits
Tag	2
Cache Index	3
Word Offset	2
Byte Offset	2

3.2 Number of Cache Tag Bits

The cache stores **8 cache lines**, requiring **3 index bits**. Each cache line stores **16 bytes**, requiring **4 offset bits**. Since the effective physical address is **9 bits**, the remaining bits are used as the cache tag.

Calculation

$$Tag = 9 - 3 - 4 = 2 \text{ bits}$$

Therefore, each cache line stores a **2-bit tag** for address comparison.

3.3 Tag Registers and Valid Bits

A direct-mapped cache stores one tag for each cache line. Since the cache contains **8 lines**, the design requires:

- **8 Tag Registers**
- **8 Valid Bits**

Each cache line has its own valid bit to indicate whether the stored data is valid.

Calculation

Number of Cache Lines = **8**

Therefore,

- Tag Registers = **8**
- Valid Bits = **8**

3.4 Number of Tag Comparators

Only **one cache line** can be selected for a given index in a direct-mapped cache. Therefore, only **one tag comparison** is required during each memory access.

Calculation

Selected Cache Line = **1**

Required Comparators = **1**

Thus, the design requires **one tag comparator**, making the hardware simple and reducing implementation cost.

3.5 Cache Datapath

The cache datapath consists of the following components:

- Address Decoder
- Tag Array
- Valid Bit Array
- Comparator
- Data Array
- Multiplexer
- Main Memory Interface

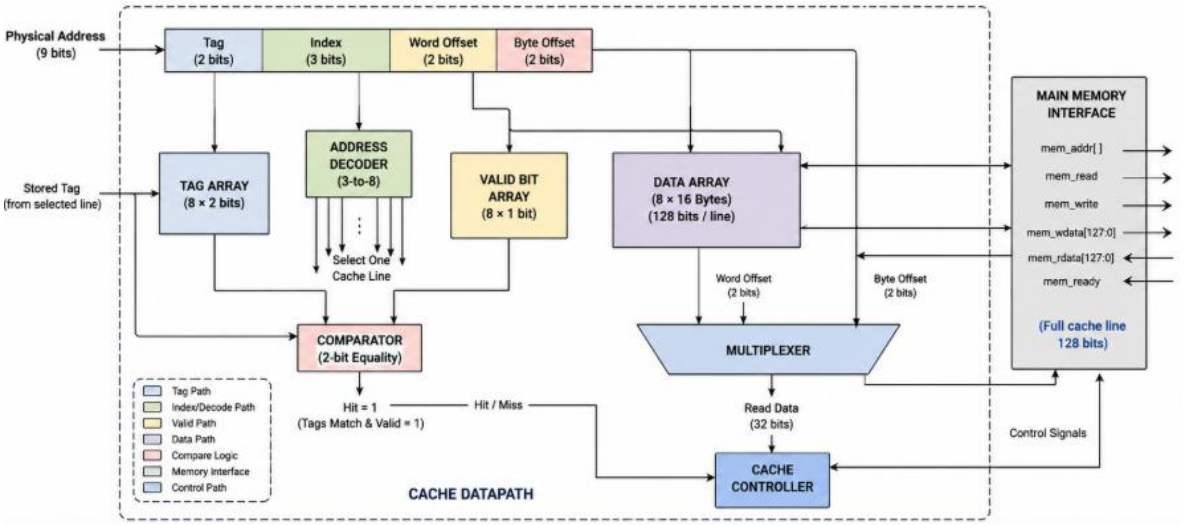
During every memory access, the cache index selects one cache line. The stored tag is compared with the incoming tag. If both tags match and the valid bit is set, a cache hit occurs and the requested word is returned to the processor. Otherwise, the controller

initiates a cache miss operation and fetches the required cache line from main memory before updating the cache.

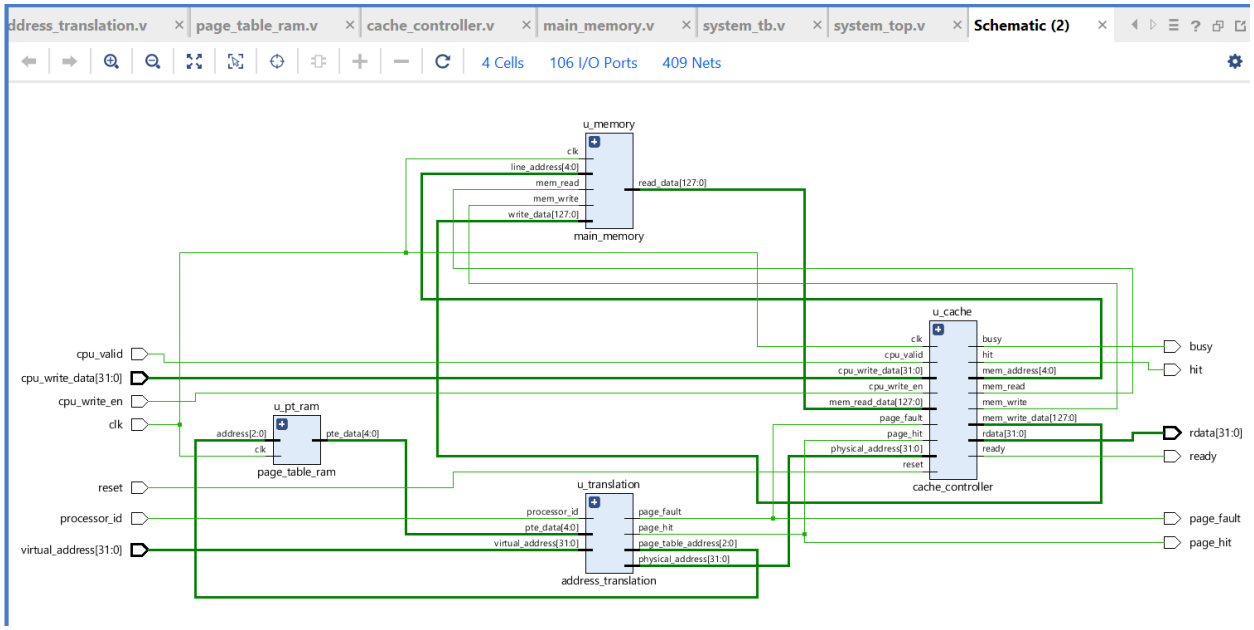
The cache datapath consists of the following components:

- Address Decoder
- Tag Array
- Valid Bit Array
- Comparator
- Data Array
- Multiplexer
- Main Memory Interface

During every memory access, the cache index selects one cache line. The stored tag is compared with the incoming tag. If both tags match and the valid bit is set, a cache hit occurs and the requested word is returned to the processor. Otherwise, the controller initiates a cache miss operation and fetches the required cache line from main memory before updating the cache.

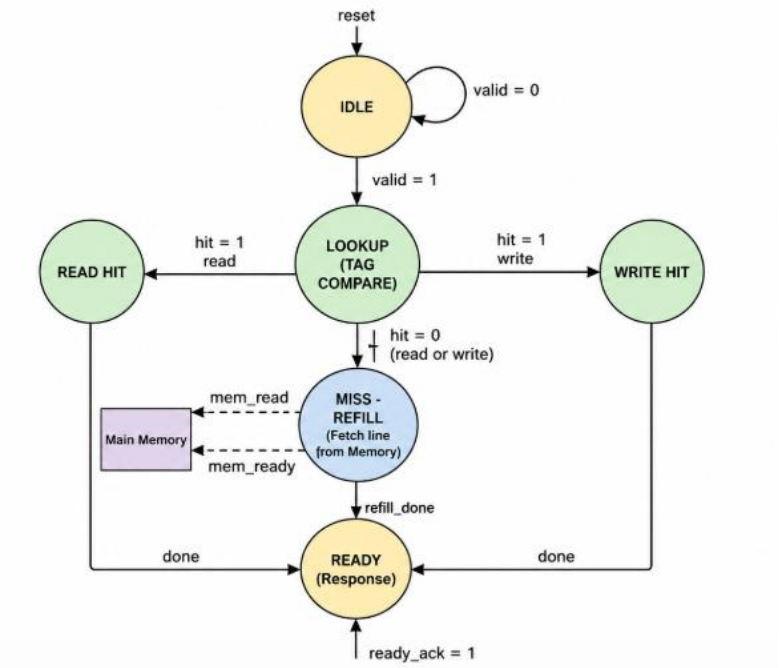


Vivado RTL schematic



3.6 Cache Controller Operation

The cache controller manages all memory access requests after address translation. It checks whether the requested data is already available in the cache by comparing the incoming tag with the stored tag of the selected cache line. If the tags match and the valid bit is set, a **cache hit** occurs. Otherwise, a **cache miss** occurs, and the required cache line is fetched from the main memory before the processor request is completed. The controller also manages the **busy** and **ready** signals to indicate the current status of the memory operation.



3.7 Read Hit Operation

During a read request, the cache controller first checks the selected cache line.

Read Hit Sequence

1. Processor issues a read request.
2. Cache index selects the corresponding cache line.
3. Stored tag is compared with the incoming tag.
4. If the tags match and the valid bit is set, a cache hit occurs.
5. The requested word is read from the cache.
6. The **ready** signal is asserted and the data is returned to the processor.

This operation requires only cache access and does not access the main memory.

3.8 Write Hit Operation

When the requested data is already available in the cache, the write operation is completed directly in the selected cache line.

Write Hit Sequence

1. Processor issues a write request.
2. Cache tag comparison is performed.
3. Cache hit is detected.
4. The selected word in the cache is updated with the new data.
5. The write operation completes and the **ready** signal is asserted.

3.9 Read Miss Operation

If the requested cache line is not available, the controller performs a cache refill.

Read Miss Sequence

1. Processor issues a read request.
2. Cache lookup is performed.
3. Tag comparison fails.
4. Cache miss is detected.
5. Main memory is accessed.
6. The required cache line is copied into the cache.
7. Tag and valid bit are updated.
8. Requested word is returned to the processor.
9. **Ready** signal is asserted.

3.10 Write Miss Operation

When a write request targets a cache line that is not currently stored in the cache, the controller first loads the required cache line from main memory before updating the data.

Write Miss Sequence

1. Processor issues a write request.
2. Cache lookup results in a miss.
3. Required cache line is fetched from main memory.

4. Cache line is updated.
5. New data is written into the selected word.
6. The write operation completes.

3.11 Signal Generation

The cache controller generates several status signals during operation.

Hit Signal

The **hit** signal becomes active when the stored cache tag matches the incoming tag and the corresponding valid bit is set.

Busy Signal

The **busy** signal remains asserted while the cache controller is processing a request, performing address lookup, or waiting for data from main memory.

Ready Signal

The **ready** signal is asserted for one clock cycle after a successful read operation, write operation, or completion of a cache refill.

Read Data (rdata)

The **rdata** output contains the requested 32-bit word and is valid only when the **ready** signal is asserted.

3.12 Cache Controller Summary

The implemented cache controller performs direct-mapped cache lookup using the translated physical address. It detects cache hits and cache misses through tag comparison, manages cache refill operations from the main memory, and generates the required control signals (**hit**, **busy**, **ready**, and **rdata**) to complete processor memory requests. The controller also operates only after successful address translation, ensuring that invalid virtual addresses do not initiate cache accesses.

Chapter 4: RTL Implementation

4.1 Overview

The complete system is implemented using Verilog HDL and consists of five main modules: **system_top**, **address_translation**, **page_table_ram**, **cache_controller**, and **main_memory**. These modules work together to perform virtual-to-physical address translation, cache lookup, and communication with the main memory.

4.2 Top Module (system_top)

The **system_top** module integrates all components of the system. It connects the processor inputs with the address translation unit, forwards the translated physical address to the cache controller, and connects the cache controller to the main memory.

Main Functions

- Connects all RTL modules.
- Transfers the translated physical address to the cache controller.
- Connects the cache controller with the main memory.
- Provides the final outputs (**hit**, **busy**, **ready**, and **rdata**) to the processor.

RTL Snippet

```
address_translation u_translation (
      .processor_id(processor_id),
      .virtual_address(virtual_address),
      .page_table_address(page_table_address),
      .pte_data(pte_data),
      .physical_address(physical_address),
      .page_hit(page_hit),
      .page_fault(page_fault)
   );

   // 2 Page Table RAM (Synchronous)
   page_table_ram u_pt_ram (
      .clk(clk),
      .address(page_table_address),
      .pte_data(pte_data)
   );

   // 3 Cache Controller (FSM and Datapath)
   cache_controller u_cache (
```

```
    .clk(clk),
    .reset(reset),
    .cpu_valid(cpu_valid),
    .cpu_write_en(cpu_write_en),
    .cpu_write_data(cpu_write_data),
    .physical_address(physical_address),
    .page_hit(page_hit),
    .page_fault(page_fault),
    .hit(hit),
    .busy(busy),
    .ready(ready),
    .rdata(rdata),
    .mem_read(mem_read),
    .mem_write(mem_write),
    .mem_address(mem_address),
    .mem_write_data(mem_write_data),
    .mem_read_data(mem_read_data)
);

// 4 Main Memory (Synchronous)
main_memory u_memory (
    .clk(clk),
    .mem_read(mem_read),
    .mem_write(mem_write),
    .line_address(mem_address),
    .write_data(mem_write_data),
    .read_data(mem_read_data)
);
```

4.3 Address Translation Module (address_translation)

The **address_translation** module converts the processor's virtual address into a physical address. It extracts the Virtual Page Number (VPN) and Page Offset, selects the appropriate PTBR according to the processor ID, accesses the page-table RAM, and generates the physical address.

Main Functions

- Extract VPN and Page Offset.
- Select PTBR.
- Generate PTE address.
- Produce Page Hit/Page Fault.
- Generate Physical Address.

RTL Snippet

```
assign vpn = virtual_address[6:5];
assign page_offset = virtual_address[4:0];
assign page_table_address = selected_ptbr + vpn;
```

4.4 Page Table RAM (page_table_ram)

The **page_table_ram** module stores the page tables used during address translation. It contains eight Page Table Entries (PTEs), representing two page tables with four entries each. Every PTE stores a valid bit and a Physical Frame Number (PFN).

Main Functions

- Store both page tables.
- Read Page Table Entries.
- Output Valid bit and PFN.

4.5 Cache Controller (cache_controller)

The **cache_controller** is responsible for cache lookup, hit detection, cache refill, and communication with the main memory. It compares the incoming tag with the stored tag of the selected cache line and performs the appropriate read or write operation.

Main Functions

- Cache lookup.
- Tag comparison.
- Cache hit detection.
- Cache miss handling.
- Cache refill from main memory.
- Generate **hit**, **busy**, **ready**, and **rdata** signals.

RTL Snippet

```
if(valid_bit[index] && tag_array[index] == tag)
   hit <= 1'b1;
else
   hit <= 1'b0;
```

4.6 Main Memory (main_memory)

The **main_memory** module stores all memory blocks and supplies a complete cache line whenever a cache miss occurs. During a cache refill, the requested cache line is transferred from the main memory to the cache controller.

Main Functions

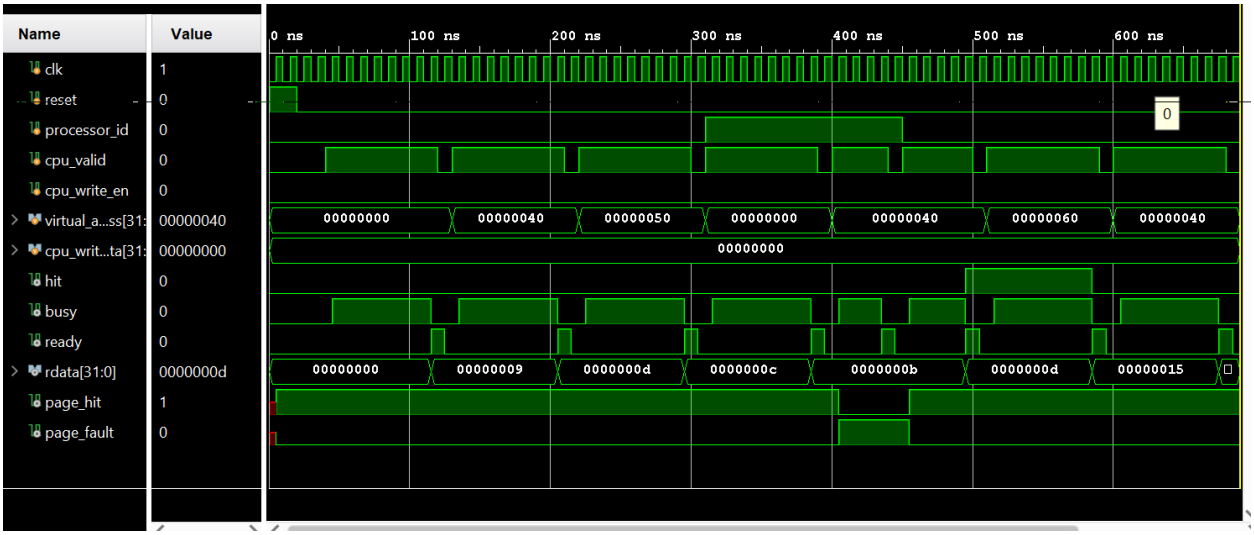
- Store 32 memory lines.
- Provide cache lines during cache misses.
- Support read and write operations.

4.7 Summary

The implemented RTL design follows a modular architecture in which each module performs a dedicated task. The address translation module converts virtual addresses into physical addresses, the page-table RAM stores the translation information, the cache controller performs cache operations, and the main memory supplies data during cache misses. The top module integrates these components into a complete cache system that satisfies the project requirements.

Chapter 5: Simulation Results and Discussion

5.1 Behavioral Simulation



The behavioral simulation verifies the functionality of the RTL design before synthesis.

Observation 1 – Valid Address Translation

At the beginning of the simulation, **processor_id = 0** accesses valid virtual addresses. The **page_hit** signal remains high while **page_fault** stays low, confirming that the selected Page Table Entries are valid. This verifies that the address translation unit correctly translates virtual addresses into physical addresses before cache access.

Observation 2 – Cache Miss and Refill

During the first access to a new address, the **busy** signal becomes high while **ready** remains low. This indicates that the requested cache line is not available and the cache controller is fetching data from main memory. Once the refill is complete, **ready** is asserted and valid data appears on **rdata**. This verifies the correct implementation of the cache refill operation.

Observation 3 – Cache Hit

When the same address is accessed again, **busy** is asserted for a shorter duration and the requested data is returned immediately on **rdata**. This shows that the data is now available in the cache, confirming correct cache-hit operation.

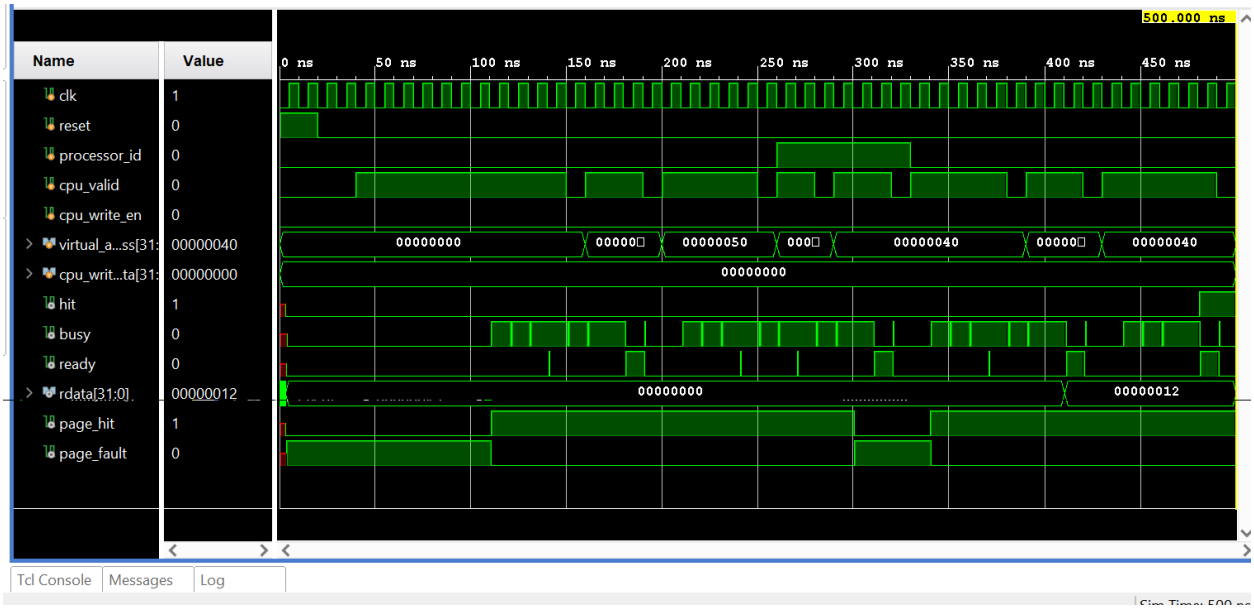
Observation 4 – Page Fault

Later in the simulation, **processor_id** changes to access an invalid page-table entry. At this point, **page_fault** becomes high and **page_hit** becomes low. No cache lookup is performed during this period, confirming that invalid pages are correctly blocked before reaching the cache.

Observation 5 – Data Verification

The waveform shows different values appearing on **rdata** during successive memory accesses. These values correspond to the requested memory locations, confirming that the cache controller correctly returns the requested data after address translation and cache lookup.

5.2 Post-Synthesis Timing Simulation



The post-synthesis timing simulation verifies that the synthesized hardware behaves the same as the RTL design.

Observation 1 – Functional Correctness

The sequence of operations remains identical to the behavioral simulation. Address translation is completed before cache lookup, cache misses trigger memory access, and cache hits return data correctly.

Observation 2 – Timing Delays

Compared with the behavioral simulation, small propagation delays can be observed because the synthesized netlist includes actual logic gates and routing delays. These delays do not affect the functional behavior of the design.

Observation 3 – Signal Verification

The **page_hit**, **page_fault**, **busy**, **ready**, and **rdata** signals behave as expected throughout the simulation. Their behavior matches the RTL simulation, confirming that synthesis has preserved the intended functionality.

5.3 Result Validation

The simulation results validate the proposed design by confirming that:

- ✓ Virtual addresses are translated correctly using the page table.
- ✓ Cache lookup starts only after successful address translation.
- ✓ Invalid pages generate a **page_fault** and prevent cache access.
- ✓ Cache misses correctly fetch data from main memory.
- ✓ Cache hits return data directly from the cache.
- ✓ Behavioral and post-synthesis simulations produce identical functional results.

These observations demonstrate that the implemented cache controller and address translation unit satisfy the functional requirements specified in the assignment.

Conclusion

This project successfully implemented a **direct-mapped cache controller** integrated with a **page-table based address translation unit** using Verilog HDL. The design performs virtual-to-physical address translation before every cache access, ensuring that only valid physical addresses are used for memory operations. Page hits and page faults are correctly detected, while the cache controller performs cache lookup, cache refill, and data retrieval from the main memory as required.

The implementation was verified using both **behavioral** and **post-synthesis timing simulations**. The simulation results confirmed correct address translation, cache hit and cache miss detection, page fault handling, cache refill operations, and successful data transfer between the cache and main memory. The synthesized design produced the same functional behavior as the RTL simulation, confirming the correctness of the implementation.

Overall, the developed system satisfies the functional requirements of the assignment and demonstrates the interaction between virtual memory management and a direct-mapped cache controller in a simplified processor memory hierarchy